

Intel PIN & TinyTracer: A Practical Guide to Dynamic Instrumentation



Hello, my name is...

- Real job: Security Researcher
- Hobby: Computer Security (again?), Hardware, books and VideoGames
- OSS contributor and author:
 - <https://github.com/cecio/>
- Sometimes Speaker (DEFCON, Insomni'hack, NullCon, etc)

PIN?

- Pin's default JIT mode is a mini-VM plus JIT compiler; it rewrites entire traces in its private code cache and runs them there
- No IAT patching, no trampoline patching, no classic hooking
- It works for Windows, Linux and Mac

TinyTracer?

https://github.com/hasherezade/tiny_tracer

- Probably the most complete example (not really an example) available for PIN usage, bridging ease of use and powerful tracing

A Pin Tool for tracing:

- API calls, including [input and output of selected functions](#)
- [defined local functions](#)
- selected instructions: [RDTSC](#), [CPUID](#), [INT](#)
- [inline system calls, including parameters of selected syscalls](#)
- transition between sections of the traced module (helpful in finding OEP of the packed module)
- [executed instructions in defined code fragments](#)

Small Demo

Simple PIN Tool for Code Coverage

- Register an instrumentation callback (e.g., for instructions or basic blocks or function entry/exit)
- Maintain coverage data (use a simple map, counters, or file output)
- Finalize and print coverage results

int main

```
int main(int argc, char* argv[])
{
    // Initialize symbol processing (needed for IMG routines, etc.)
    PIN_InitSymbols();

    // Initialize PIN
    if (PIN_Init(argc, argv))
    {
        std::cerr << "[!] PIN Initialization failed." << std::endl;
        return 1;
    }
}
```

int main

```
// Register instrumentation callbacks
IMG_AddInstrumentFunction(OnImageLoad, nullptr);
TRACE_AddInstrumentFunction(OnTrace, nullptr);
PIN_AddFiniFunction(OnFini, nullptr);

// Start the program (never returns)
PIN_StartProgram();
return 0;
```


OnImageLoad

```
static VOID OnImageLoad(IMG img, VOID* v)
{
    if (IMG_IsMainExecutable(img))
    {
        imageBase = IMG_LowAddress(img);
        std::cerr << "[+] Main Executable loaded at 0x"
                    << std::hex << imageBase << std::endl;
    }
}
```

OnTrace

```
static VOID OnTrace(TRACE trace, VOID* v)
{
    for (BBL bbl = TRACE_BblHead(trace); BBL_Valid(bbl); bbl = BBL_Next(bbl))
    {
        BBL_InsertCall(bbl,
                       IPOINT_ANYWHERE,
                       (AFUNPTR)RecordBasicBlock,
                       IARG_ADDRINT, BBL_Address(bbl),
                       IARG_END);
    }
}
```

Traces usually begin at the target of a taken branch and end with an unconditional branch

RecordBasicBlock

```
static VOID RecordBasicBlock(ADDRINT addr)
{
    if (imageBase != 0)
    {
        ADDRINT rva = addr - imageBase;
        coveredRvas.insert(rva);
    }
}
```

OnFini

```
static VOID OnFini(INT32 code, VOID* v)
{
    CoverageFile.open("coverage_rva.out");
    if (!CoverageFile.is_open())
    {
        std::cerr << "[!] Could not open coverage_rva.out for writing!\n";
        return;
    }

    for (auto rva : coveredRvas)
    {
        CoverageFile << "CCu [COVERED] @0x" << std::hex << rva << std::endl;
    }

    CoverageFile.close();
    std::cerr << "[+] Code coverage saved to coverage_rva.out" << std::endl;
}
```

Small Demo

Advanced Use Cases

Security-Focused Instrumentation

Malware analysis, binary hardening, fuzzing coverage

Performance Profiling

Performance hotspots, dynamic optimization, cache modeling

Debugging

Advanced code coverage, Deterministic record/replay (to reproduce Heisenbugs)

References

- PIN:
<https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-binary-instrumentation-tool-downloads.html>
- TinyTracer:
https://github.com/hasherezade/tiny_tracer/tree/master
- CodeCov:
<https://gist.github.com/cecio>
<https://github.com/cecio/talks-and-rants/tree/master/ComeToCode-2025>

Thank you!
